

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I JASMITHA R(192524295) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

JASMITHA R

Annexure A

Reverse Engineering

1. Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

2. Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

4. Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q1. Reverse Engineering a Recommendation System

System Overview

A large e-commerce recommendation engine ingests continuous streams of clickstream, purchase, cart, and browsing data, and returns product suggestions within milliseconds. Reverse engineering this black box means

inferring its internal pipeline purely from observed inputs (user behaviour) and outputs (ranked recommendations).

Inferred Architecture (PySpark + RDD based pipeline)

- **Data Ingestion Layer:** Clickstream events, purchase logs, and product-catalogue updates are pushed into a message queue (e.g., Kafka) and consumed by Spark Streaming, which converts each micro-batch into RDDs (DStream → RDD per batch).
- **Data Cleaning & Transformation:** map() and filter() transformations remove bot traffic, duplicate events, and null sessions; flatMap() explodes multi-item cart events into individual (user, item, action) tuples.
- **Feature Construction:** reduceByKey() and groupByKey() aggregate user–item interaction counts, dwell time, and recency to build implicit feedback vectors; join() merges behavioural RDDs with a product-metadata RDD (category, price, tags).
- **Model Layer:** The aggregated RDDs feed a collaborative-filtering / matrix-factorisation model (e.g., Spark MLlib ALS) that is periodically retrained in batch mode, while a lightweight similarity lookup handles real-time re-ranking.
- **Serving Layer:** Precomputed recommendation lists are cached (e.g., in Redis) and served through an API; only the ranking/re-ranking step happens online to keep latency low.

Role of Intelligent Agents

- **Monitoring Agent:** continuously watches click-through and conversion rates on served recommendations.
- **Learning/Update Agent:** triggers incremental retraining or re-weighting of the model when engagement metrics drop, closing the feedback loop between user response and model parameters.
- **Personalization Agent:** at request time, selects/re-ranks candidates for a specific session using context (device, time of day, current cart) — an example of a reflex-with-state agent acting on the RDD-derived feature store.
- **Coordinator Agent:** schedules distributed batch jobs (feature refresh, model retrain) and manages fault recovery via Spark's RDD lineage (recomputation instead of replication).

Inference Summary

The system is almost certainly a lambda-architecture design: a batch layer (PySpark RDD transformations for heavy aggregation and model training) paired with a fast serving layer, coordinated by autonomous agents that monitor performance and trigger retraining — explaining both the accuracy (batch-trained models) and the real-time responsiveness (cached, agent-updated recommendations).

Q2. Reverse Engineering a Smart Traffic Management System

System Overview

The system continuously adjusts signal timings across a city based on live vehicle density and congestion, implying a low-latency streaming pipeline with distributed decision-making at each intersection.

Inferred Data Pipeline (RDD-based Streaming)

- **Sensor Ingestion:** IoT sensors, cameras, and GPS feeds from vehicles stream data continuously; Spark Streaming discretises this into micro-batch RDDs (typically every 1–5 seconds) per intersection or zone.

- **Transformation:** `map()` converts raw sensor readings into structured (`intersection_id`, `timestamp`, `vehicle_count`, `avg_speed`) records; `filter()` discards corrupted or out-of-range sensor readings.
- **Aggregation:** `reduceByKey()` on `intersection_id` computes rolling congestion indices; window operations (windowed RDD/DStream) compute short-term trends (e.g., last 2 minutes) to distinguish transient jams from sustained congestion.
- **Cross-Intersection Correlation:** `join()` combines neighbouring-intersection RDDs so the system can detect upstream congestion propagating toward a signal before it arrives locally.

Agent Interactions

- **Local Signal Agents:** one per intersection, each a reflex agent that reads its own RDD partition of live density data and adjusts green-light duration within safety-bounded limits.
- **Coordinating/Regional Agent:** aggregates neighbouring local agents' states to implement 'green wave' synchronisation along a corridor, resolving conflicts when two agents' locally-optimal decisions clash.
- **Predictive Agent:** uses short-horizon forecasts (from windowed RDD aggregates) to pre-emptively adjust timings ahead of predicted congestion rather than reacting after the fact.

Decision-Making Under Time Constraints

- Decisions are made on micro-batches rather than waiting for full-city convergence — an approximate, low-latency answer is prioritised over a globally optimal one.
- In-memory RDD caching (`persist()/cache()`) avoids repeated disk I/O so each decision cycle completes within the signal's real-time deadline.
- RDD lineage enables fast recovery from node failure without re-ingesting raw sensor data, preserving continuity of control during hardware faults.
- A fallback reflex mode (fixed-time signal plan) is likely triggered if the agent does not receive fresh data within its decision window, ensuring safety even when the pipeline stalls.

Q3. Reverse Engineering a Fraud Detection Pipeline

System Overview

Millisecond-level flagging of fraudulent transactions implies a hybrid architecture: an online scoring path for instant decisions and an offline PySpark/RDD pipeline for feature engineering and model retraining at scale.

Inferred Distributed Pipeline

- **Ingestion:** Every transaction event (amount, merchant, location, device fingerprint, timestamp) is streamed in; Spark Streaming batches these into RDDs at sub-second intervals.
- **Feature Extraction:** `map()/flatMap()` derive engineered features per transaction — transaction velocity (count in last N minutes via `reduceByKey` on `account_id`), deviation from historical spending pattern, geo-distance from last known location, merchant-risk score (via `join()` with a merchant-risk RDD).
- **Anomaly Detection:** The feature RDDs are scored by a pre-trained model (e.g., logistic regression, gradient-boosted trees, or an autoencoder trained offline in Spark MLlib on historical labelled/unlabelled RDDs); anomaly scores above a threshold are flagged.

- **Batch Retraining:** Historical transaction RDDs are periodically reprocessed (groupByKey/reduceByKey for aggregate fraud statistics) to retrain the model and recalibrate thresholds as fraud patterns evolve.

Agent-Based Alert Mechanism

- **Scoring Agent:** sits directly in the transaction path, applies the current model to the incoming feature vector, and returns approve/hold/decline within the millisecond budget.
- **Escalation Agent:** when a transaction is borderline (score near the decision threshold), routes it to a secondary, more expensive rule-check or holds it for human/analyst review rather than an automatic decline.
- **Feedback Agent:** ingests confirmed fraud/non-fraud outcomes (chargebacks, customer disputes) back into the training RDDs, closing the loop and reducing false positives over time.
- **Coordination Agent:** manages load balancing across distributed scoring nodes and ensures RDD partitioning is keyed by account/card so related transactions are evaluated with consistent recent-history context.

Why This Design Fits the Observed Behaviour

Millisecond response times rule out a purely batch approach for the scoring decision itself, while the system's evident adaptability to evolving fraud patterns indicates a continuously retrained model — consistent with a Spark-based lambda architecture where RDD transformations handle heavy feature aggregation offline and a lightweight agent performs the final real-time scoring.

Q4. Reverse Engineering a Personalized Learning System

System Overview

The platform adapts content difficulty based on performance and engagement, implying continuous tracking of learner interactions, RDD-based aggregation of performance history, and an adaptive agent that alters the learning path.

Inferred Data Processing (RDD Transformations)

- **Event Capture:** Every learner action — quiz attempt, time-on-task, video pause/replay, hint usage — is logged as an event and loaded into Spark as an RDD of (student_id, activity_id, response, correctness, timestamp) tuples.
- **Cleaning & Structuring:** map()/filter() normalise scores and discard incomplete/abandoned sessions.
- **Performance Aggregation:** reduceByKey() on student_id computes rolling accuracy, average response time, and streak/mastery indicators per topic; groupByKey() organises records by topic to compute topic-level difficulty statistics across the whole cohort.
- **Engagement Modelling:** join() combines performance RDDs with engagement RDDs (login frequency, session length) to build a holistic learner-state vector used to decide not just what content, but how much and how often, to present.

Role of Intelligent Agents in Adaptive Learning

- **Diagnostic Agent:** evaluates the learner-state vector after each interaction to classify mastery level per concept (e.g., struggling, on-track, advanced).

- **Content-Selection Agent:** a goal-based agent that selects the next item's difficulty to keep the learner in an optimal challenge zone — increasing difficulty after streaks of correct answers, easing back after repeated errors or long hesitation.
- **Feedback Agent:** generates real-time hints, explanations, or remediation content, personalising the message based on the specific error pattern detected in the RDD-derived history rather than a generic response.
- **Cohort-Learning Agent:** periodically uses the aggregated, cross-student RDDs to identify which content items are globally too easy/hard and adjusts baseline difficulty calibration for future learners.

Inference Summary

The adaptivity observed (content difficulty shifting with performance) is best explained by a closed loop: RDD-based batch/streaming aggregation continuously updates each learner's mastery profile, and a goal-driven content-selection agent consumes that profile in real time to personalise the next learning step, with periodic cohort-level RDD analysis recalibrating the system as a whole.

Q5. Reverse Engineering a Cloud-Based AI Monitoring System

System Overview

The platform automatically scales resources and detects anomalies in real time, implying a distributed log-processing pipeline (PySpark) feeding autonomous scaling and alerting agents, with fault tolerance built into the RDD execution model itself.

Inferred Log-Processing Architecture

- **Log Ingestion:** Metrics and logs (CPU, memory, latency, error rates) from thousands of containers/VMs are streamed continuously; Spark Streaming converts them into micro-batch RDDs partitioned by service/host.
- **Parsing & Filtering:** `map()` parses raw log lines into structured metric records; `filter()` removes noise (health-check pings, duplicate heartbeats).
- **Aggregation:** `reduceByKey()` computes per-service rolling averages and percentiles (e.g., p95 latency) over sliding windows; `groupByKey()` clusters related events (same trace/request ID) for root-cause correlation.
- **Anomaly Scoring:** The aggregated RDDs are compared against learned baselines (statistical thresholds or a trained model) to flag deviations — sudden latency spikes, error-rate surges, or resource saturation.

How RDD Operations Support Fault Tolerance

- RDD lineage (the DAG of transformations) allows any lost partition to be recomputed from source data rather than requiring replicated storage, so the monitoring pipeline itself survives node failures.
- Immutability of RDDs means retried/recomputed tasks always produce identical results, which is essential when the same log stream might be reprocessed after a failure.
- `persist()/cache()` on frequently reused aggregate RDDs (e.g., current baselines) reduces recomputation cost and keeps anomaly detection within real-time bounds even during partial failures.

Coordination of Monitoring, Scaling, and Anomaly Response

- **Monitoring Agent:** continuously evaluates the anomaly-scored RDDs and raises alerts when metrics cross thresholds.
- **Scaling Agent:** a utility-based agent that decides how many additional container/VM instances to provision (or de-provision) based on aggregated load trends, balancing cost against performance SLAs.
- **Anomaly-Response Agent:** on detecting a genuine anomaly (not just load growth), triggers remediation — restarting a faulty service, rerouting traffic, or paging an on-call engineer — distinguishing itself from the scaling agent, which reacts to load rather than faults.
- **Coordinator/Orchestration Layer:** arbitrates between the scaling and anomaly-response agents when their actions could conflict (e.g., avoiding a scale-down while an anomaly investigation is in progress).

Inference Summary

The combination of real-time responsiveness and resilience to failure points to a Spark-based streaming pipeline where RDD lineage provides fault tolerance 'for free', while a set of specialised agents (monitoring, scaling, anomaly-response) operate on top of the continuously aggregated RDDs to keep the cloud platform both elastic and self-healing.